

Runtime state, checkpoints, and provenance

ar089 · 14 August 2026 · Draft

Contents

1. Developer guide
2. Runtime state
3. Checkpoints
4. Provenance
5. API reference

Developer guide

Runtime state

Runtime state contains the dynamic values required to continue one trajectory: voltages, refractory counters, conductances, projection and input histories, and completed step count. It excludes static parameter values.

A runtime-state signature covers timestep, populations, input shapes, projections, delays, synapses, and state layout. Incompatible state is rejected rather than partially loaded. This supports burn-in followed by a compatible intervention without restarting the circuit.

Runtime-state save, load, validation, and continuation are implemented.

Checkpoints

Parameter checkpoints and runtime state serve different purposes. A parameter checkpoint describes learned or initialized weights. Runtime state describes where one simulation trajectory currently is. A complete training checkpoint additionally requires optimizer state, data-order state, and stochastic-stream state.

Parameter loading and graph runtime state exist today. A portable graph-training checkpoint and exact optimizer resume contract are not implemented.

Provenance

A run should retain the bundle digest, resolved execution protocol, seeds, backend and version, realized initializer statistics, checkpoint identity, and produced artifacts. Dense-array graph runs now emit a versioned execution protocol in `metrics.json`, including source digests, resolved input contracts, dataset metadata, timing, masks, and execution seed. A resumed run must preserve optimizer state, data order, and stochastic streams when exact continuation is claimed.

The bundle is network provenance, not the complete experiment record. Experiment runners still own conditions, orchestration, analysis, figures, and publication.

API reference

GraphRuntimeState

```
GraphRuntimeState(  
    signature,  
    compatibility,  
    completed_steps,  
    voltages,  
    refractory,  
    conductances,  
    population_histories,  
    input_histories,  
)
```

Every tensor group is keyed by stable graph identifier. `.detached(device="cpu")` returns a cloned, detached state on the requested device. Static parameters are not included.

Save, load, and compatibility

```
save_runtime_state(path, state) -> Path  
load_runtime_state(path, *, device="cpu") -> GraphRuntimeState  
runtime_state_compatibility(plan) -> dict  
runtime_state_signature(plan) -> str
```

Runtime state uses schema `tools/snn.graph-runtime-state/v1`. Loading verifies the manifest and tensor payload. Simulation compares the stored signature and compatibility structure with the current graph plan before restoring any state.

Resume execution

```
result = simulate(spec, runtime_state=previous.runtime_state)
```

A resume request must preserve graph structure, timestep, population sizes, projections, delays, synapses, parameter shapes, batch shape, and state dtypes.

`result.runtime_state.completed_steps` reports the cumulative trajectory length.

Provenance fields

Graph simulation metrics include device, recording profile, build and simulation timing, runtime-state schema and signature, completed steps, and the resolved execution protocol. Dense file bindings add source paths, SHA-256 digests, array keys, shapes, dtypes, dataset metadata, timing, masks, and execution seed.

Next: Compatibility, status, and extension